EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

Schedule

1. One request, followed through Aider
2. Reading failures as evidence
3. Hackathon 1 briefing
4. The build: scope, ownership, rubric
5. Where agentic tools go next
6. Office hours

WHAT IT IS

Aider at a glance

Aider is a program you run in a terminal, inside a git repository.

It reads

Only the files you choose, plus a summary of the rest.

It asks

Sends text to a model over one HTTP API. Gets text back.

It writes

Edits the files, then commits, one commit per edit set.

Not an editor plugin, not a web chat, not a service. A local process with your repository open.

WHAT IT NEEDS

Models and endpoints

Model	`qwen3.5:4b` or `qwen3.5:9b`. The 9B is recommended
Endpoint	Anything serving the OpenAI-compatible Chat Completions API
Hosting	Your machine or CAEN. Ollama is one option, not a requirement

```
model: openai/qwen3.5:9b
```

The `openai/` prefix selects the *protocol*, not the vendor.

Changing where a model is served does not change which models you may use.

The three config files

File	Holds
`.env`	`OPENAI_API_BASE`, `OPENAI_API_KEY`. Where the endpoint is
`.aider.conf.yml`	Model, edit format, auto-commit. How Aider behaves
`.aider.model.settings.yml`	Per-model settings, including edit format and repo map

```yaml
- name: openai/qwen3.5:9b
  edit_format: diff
- name: openai/qwen3.5:4b
  edit_format: whole
```

The 4B rewrites whole files. The 9B sends diffs. Same request, different reply shape.

Starting a session

```
$ cd ~/taskr
$ aider
Aider v0.86.2
Model: openai/qwen3.5:9b with diff edit format
Git repo: .git with 24 files
Repo-map: using 1024 tokens
>
```

Four things the banner tells you, and all four are worth a glance: version, model, edit format, and that it found your repository.

RUNNING IT

Aider’s command set

Command	Does what	Touches files
`/add`	Put a file in the context, editable	no
`/read-only`	Put a file in the context, citable only	no
`/drop`	Take a file back out	no
`/tokens`	Break the budget down by source	no
`/ask`	Question about the code, no edits proposed	no
`/code`	Ask for an edit, the default mode	**yes**
`/diff`	Show what the last exchange changed	no
`/undo`	Revert Aider’s last commit	**yes**

THE RUNNING EXAMPLE

One request, three files

```
Add a --priority flag to taskr's add command.
```

taskr/cli.py

Accepts the flag and its default.

taskr/task.py

Carries the value on a task.

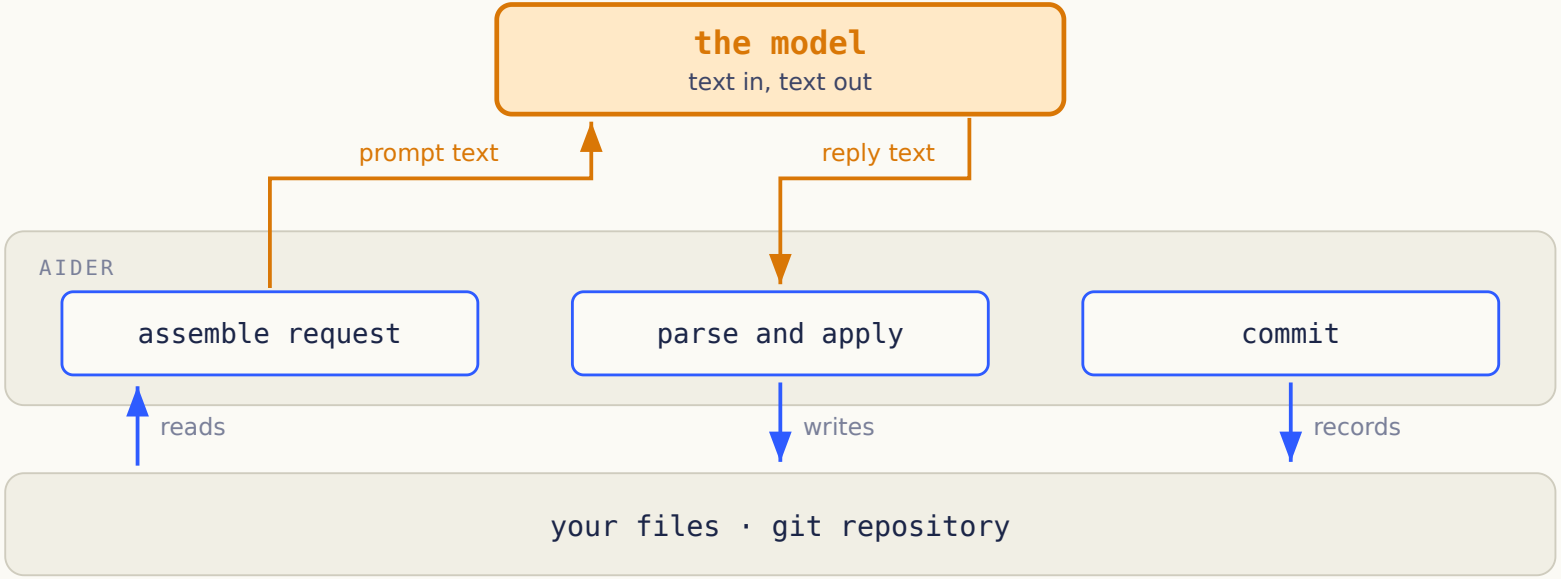
taskr/store.py

Writes it down and reads it back.

One turn

SELECTED FILES TO COMMIT

Aider, the model, and your repo



No line runs from the model to your repository. You pick the task and judge the result.

What goes into the request

- The instructions that define the edit format
- The files you added, in full
- A repository map, when it is enabled
- The chat history from this session
- Your new message

Context is a selection. A file on disk that you never added is not in it.

File selection with /add

```
/add taskr/cli.py taskr/task.py taskr/store.py
/read-only specs/priority.md
/tokens
```

Added files can be rewritten. Read-only files can only be cited.

Naming a path in a sentence does not load it. Only these commands do.

The repo map

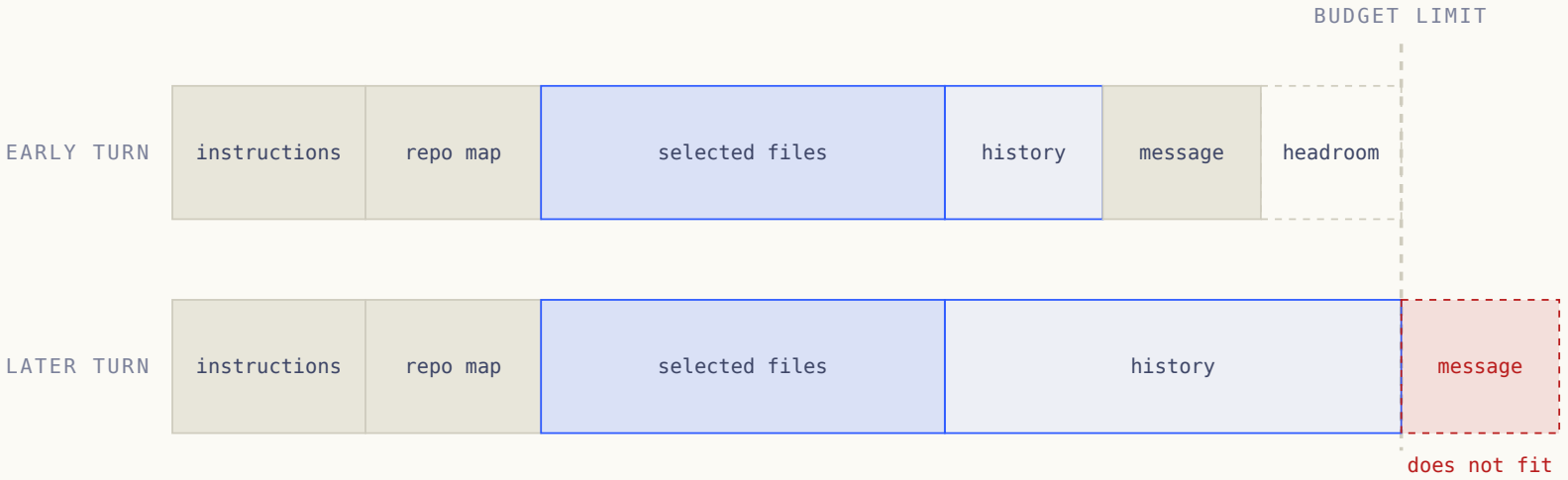
```
taskr/store.py
  Store.add(title, tags) -> Task

taskr/task.py
  Task: id, title, tags
```

Aider builds a graph of definitions and references, then ranks the useful parts into a token budget.

A signature tells you where to look. It does not tell the model how the function works.

STEP 1 · BUDGET

One shared token budget



History and the repo map spend the same budget as the file you actually care about.

Edit-shaped replies

```
taskr/cli.py
<<<<<<< SEARCH
add.add_argument("title")
=======
add.add_argument("title")
add.add_argument("--priority", default="normal")
>>>>>>> REPLACE
```

This block adds a flag to the CLI. Nothing stores the value and nothing validates it.

The parse step

Parsing asks: is this a block?

A missing divider is a syntax error. Report it and keep running.

Validation asks: does it apply?

An unknown path, or zero exact matches, fails later and for a different reason.

Two questions, two error messages. A student who merges them writes one useless message for both.

STEP 4 · APPLY

Exact-match application

On disk

```
add.add_argument("title")
```

In the SEARCH block

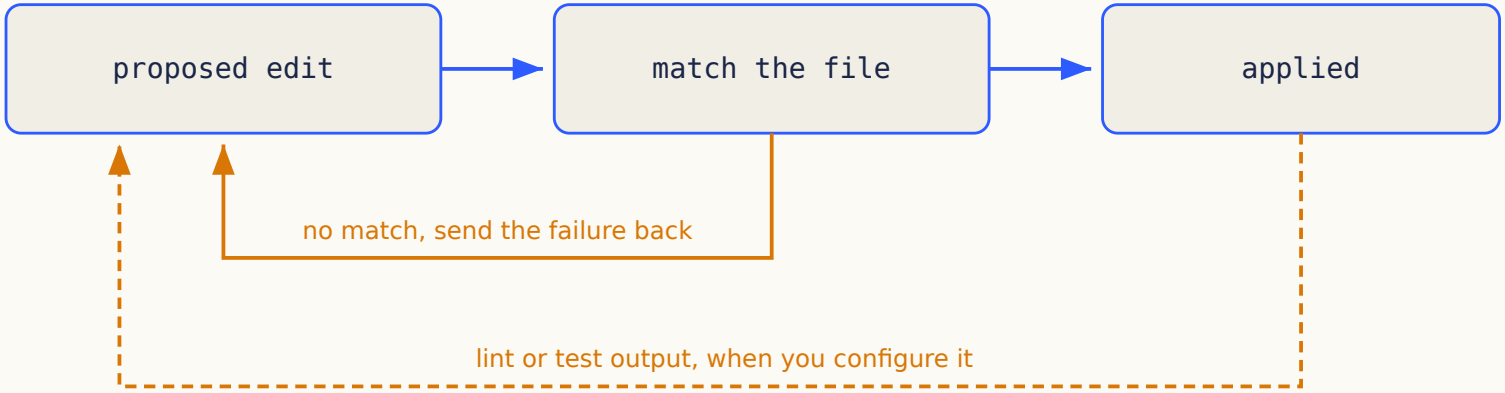
```
add.add_argument( "title" )
```

Two extra spaces. Zero matches. The block is well formed and cannot be applied.

Aider can fall back to looser matching, and it can keep the blocks that did apply.

Your Stage 1 contract does neither. Exact matches only, and all blocks or none.

The repair loop



Aider chooses the next action here from a fixed set. It is a repair cycle, not a tool the model picked.

One commit per edit set

```
* 8f2a1c3  aider: add --priority to the add command
* 41b9e07  aider: store priority on Task
* 0c7d5e1  initial taskr
```

Undo means resetting to the parent of a commit your own session created.

Git reverses file changes. It does not reverse an email that has already been sent.

CHECK · 3 MINUTES

The six steps, recapped

For `--priority`, answer with the person next to you:

1. What entered the context, and what did not?
2. Which program read the reply?
3. What could look correct and still be wrong?
4. What evidence would show the feature works?

Diagnosis

READ THE FAILURE BEFORE YOU RE-PROMPT

DIAGNOSIS

Three failure symptoms

What you see	What to check first
It calls a function that does not exist	The file defining it was never added
The block looks right and will not apply	The file changed after the model saw it
Tests pass, the feature is still broken	The tests never covered the missing part
The request times out	The endpoint URL, the served model, the input size

More than one cause can produce the same symptom. Name the evidence that separates them.

DIAGNOSIS

Better results, same model

A chat window

You paste code in. It answers with code. You copy the answer back and hope you pasted the right version.

Aider, same model

It reads the files you selected, answers in a fixed edit format, applies the result and commits it.

The model still has limits. You can measure them once the harness stops being the variable.

PRACTICE · 6 MINUTES

A feature that half works

```
$ taskr add "ship lab" --priority high
added #4
$ taskr list
#4  ship lab   priority: high

$ taskr list        # new process, same database
#4  ship lab   priority: normal
```

What do you inspect first? Write the acceptance criterion that was missing, and the test that would have caught it.

THURSDAY SEPTEMBER 24

Hackathon 1

- Two hours, during the session, on your own project
- The feature prompt is revealed in the room
- You submit before you leave
- Graded on its own, separately from the build

Bring a runnable increment. There is no separate hackathon project to prepare.

The build itself is due the next day, **Friday September 25, 11:59 PM**.

The build

DESIGN FIRST, THEN IMPLEMENT IN INCREMENTS

What the packet contains

In the packet

SPEC.md, RUBRIC.md, ACCEPTANCE.md, EXAMPLES.md, DESIGN-GUIDE.md, WORKFLOW.md, and the Aider configuration you build with.

Not in the packet

Application code. Tests. A module layout. A conversation loop. A diagram of the answer.

You run `aider` to build it. You write `bin/assistant` to run what you built.

The assistant’s behavior

```
/files                → list selected files
/add greet.py         → include this file in model requests
What does greet do?   → stream an answer about the selected code
Change Hello to Hi.   → validate the reply and preview a diff
```

A terminal assistant over selected text files. Not all of Aider.

The approval step

Point in the interaction	`greet.py`	Git
Diff shown, waiting	Still `Hello`	No new commit
You approve	Now `Hi`	One owned edit commit
You decline	Still `Hello`	No new commit
You undo the approved edit	Back to `Hello`	Reset to the parent commit

One invalid block cancels the whole proposal, including the blocks that were fine.

SCOPE

The seven features

Feature	Required behavior
F1	Conversation, budget handling, streamed replies
F2	Add, drop, list and clear context
F3	Current file contents, rendered consistently
F4	An edit-format prompt you have tested
F5	Parsing with useful errors
F6	Exact edits, complete preview, explicit approval
F7	One commit per accepted edit set, guarded undo

CONFIGURATION

The assistant’s config schema

```yaml
base_url: https://api.example.edu/v1
model: course-model-id
api_key_env: ASSISTANT_API_KEY
context_budget: 8000
timeout_seconds: 60
temperature: 0.2
```

The schema is fixed. How you load, represent and validate it is yours.

OWNERSHIP

Architecture ownership

We specify	You design
Observable behavior and safety rules	Responsibilities and who owns state
The API and configuration contract	Internal interfaces and error flow
Acceptance scenarios and the rubric	Tests, fixtures, increment boundaries
Where the course is going	Where later capabilities enter

Different architectures can earn full marks. Designing for a future you cannot describe cannot.

SPECIFICATIONS

Two sizes of spec

System spec, written once

Behavior, architecture, interfaces, failure handling, verification, and why you chose it.

Increment spec, one per change

Scoped context, ordered tasks, and a result you can check when it is done.

Commit the design before the application code. Revise it when evidence changes a decision.

The three diagrams

Diagram	The question it answers
Components and data flow	Who owns state, and what crosses each boundary?
Request sequence	Who does what during one proposed edit?
Edit lifecycle	When can an edit be approved, rejected or undone?

Mermaid is enough. Keep the first versions in git and update the final ones to match what you built.

Rubric: the three buckets

One hundred points inside the build grade, in three groups.

Group	Points	What earns them
Specification and diagrams	40	Requirements and acceptance criteria, architecture and interfaces, Aider implementation specs, three diagrams
Behavior and verification	40	The seven features scored one at a time, behavioral tests, safety and recovery tests, integration evidence
Evidence and reconciliation	20	Spec-first history, deliberate Aider use, diagnosis and reconciliation, operating instructions, model disclosure

The per-criterion split is in `RUBRIC.md`, and that file is what you are graded against.

Model rules for specifications

- Drafting, critique, diagrams, code and tests all fall under it
- The 9B is recommended, the 4B is permitted
- Secondary models get recorded the same way
- Any compatible hosting service is allowed

A different provider does not authorize a different model.

THROUGH DECEMBER

One repository to December

APPLY
one loop, exact edits, git

ANALYZE
model-selected tools

CREATE
memory, hardening, another way in

Start with one small result you can check. Refactor when you need to, and keep the phase snapshots.

What comes next

THE SAME REQUEST, IN A DIFFERENT KIND OF TOOL

Claude Code on the same request

Aider	A general agentic CLI
You frame a bounded edit task	You hand over a broader task
Selected files and a repo map	Tools that discover and read files
A fixed repair cycle	The model picks the next tool
You check between tasks	Permissions and stop rules bound it

Harness versus instructions

- Project instructions give persistent guidance
- Skills package instructions for a particular task
- Hooks attach checks to events
- Tool integrations expose actions and results

Prose asks. Code and permissions enforce.

DESIGN FOR IT NOW

Boundaries for later capability

- Where could a model-selected tool join your control flow?
- Could you swap the endpoint client without touching state management?
- Could a web interface reuse the same application behavior?

Answer with a boundary, not an implementation. You are allowed to change your answer in November.

Office hours

THE REST OF THE SESSION IS YOURS

Questions?